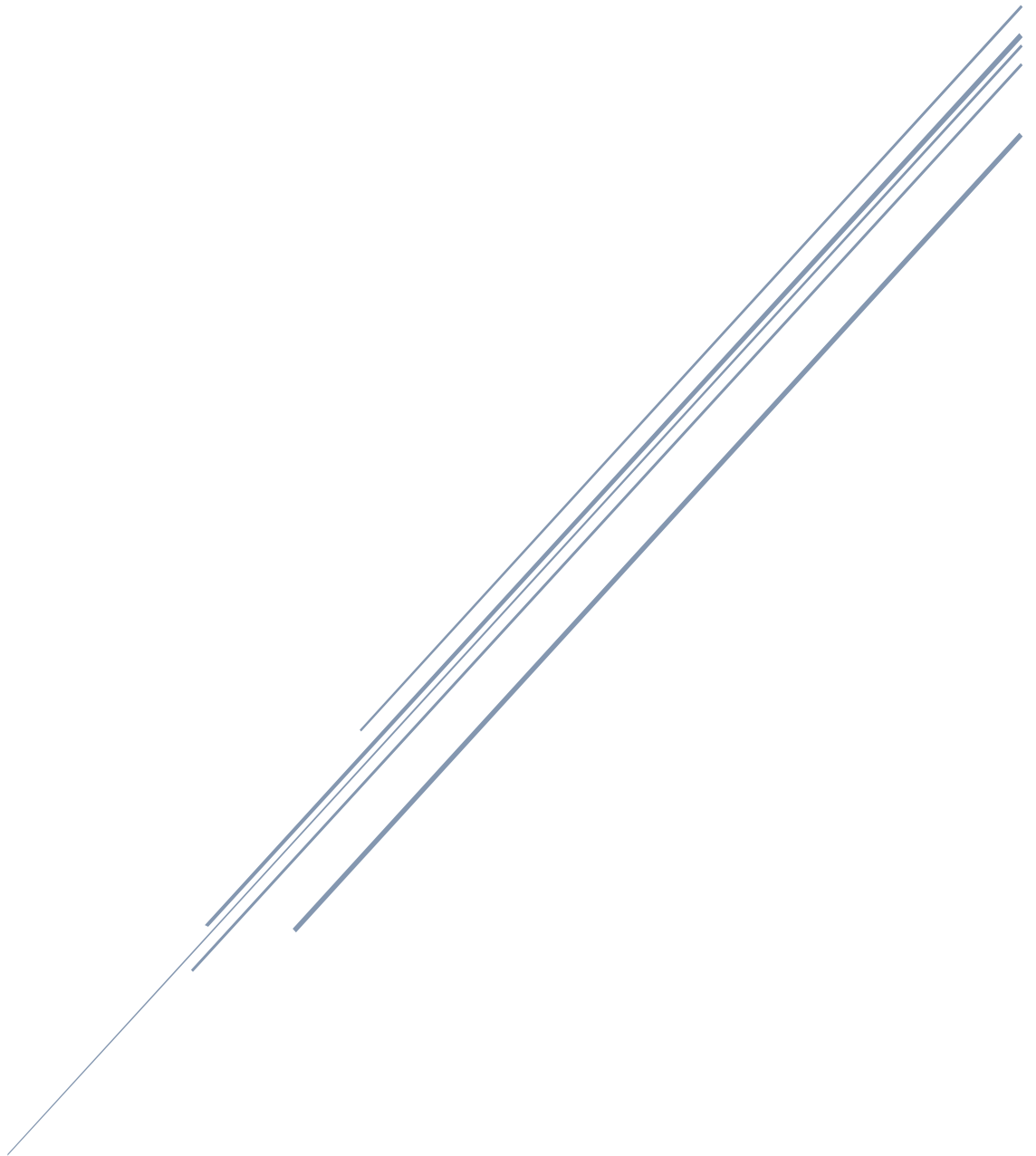


CHAPTER 3

Linear regression



3.1 Definition

Linear regression lives up to its name: it is a very straightforward approach for predicting a quantitative response Y on the basis of a predictor variable X .

We assume that the true relationship between X and Y takes the form $Y = f(X) + \epsilon$ for some unknown function f , where ϵ is a mean-zero random error term. If f is to be approximated by a linear function, then we can write this relationship as

$$Y = \beta_0 + \beta_1 X + \epsilon.$$

Here β_0 is the intercept term—that is, the expected value of Y when $X = 0$, and β_1 is the slope—the average increase in Y associated with a one-unit increase in X . The error term: the true relationship is probably not linear, there may be other variables that cause variation in Y , and there may be measurement error. We typically assume that the error term is independent of X . The model given by this equation defines the *population regression line*, which is the best linear approximation to the true relationship between X and Y .

3.2 Linear regression model

There are two very different algorithms or approaches for implementing the Linear Regression model: the “closed-form” solution approach known as the Ordinary Least Squares (OLS) method and the iterative optimization approach known as the Gradient Descent (GD).

3.2.1 Closed-form solution: Ordinary least squares (OLS)

OLS is a type of linear least squares method for estimating the unknown parameters in a linear regression model. OLS chooses the parameters of a linear function of a set of explanatory variables by the principle of least squares:

minimizing the sum of the squares of the differences between the observed dependent variable (values of the variable being predicted) in the given dataset and those predicted by the linear function.

OLS finds the optimal parameters by computing a closed-form solution for the **Normal equation**.

The OLS method is suitable for the following scenario.

- Dataset: not too large and fits into computer's memory
- No. of Features: not too large
- Relationship between input (features) and output (target): Linear

a) Simple linear regression

Example: Let's consider the simple linear regression model in which the price of a house is related to the number of square feet of living area (SQFT).

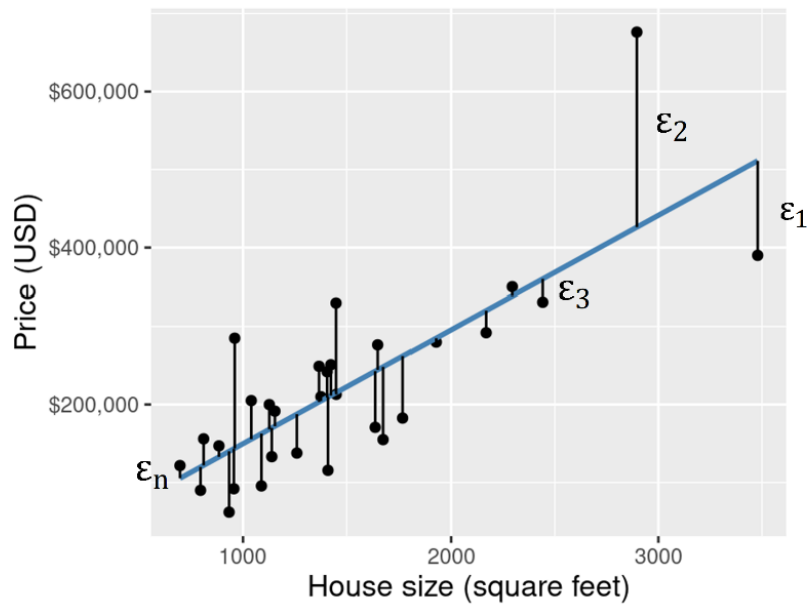


Figure 3.1 Scatter plot of sale price versus size

We have the system of equations:

$$\begin{cases} y_1 = \beta_0 + \beta_1 x_1 + \varepsilon_1 \\ y_2 = \beta_0 + \beta_1 x_2 + \varepsilon_2 \\ y_3 = \beta_0 + \beta_1 x_3 + \varepsilon_3 \\ \vdots \\ y_n = \beta_0 + \beta_1 x_n + \varepsilon_n \end{cases}$$

This system of equations can be written as:

$$\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

So, we get

$$\varepsilon = Y - X\beta$$

The L2 norm of ε is defined by

$$\|\varepsilon\| = \sqrt{\varepsilon_1^2 + \varepsilon_2^2 + \varepsilon_3^2 + \dots + \varepsilon_n^2} = \|Y - X\beta\|$$

The objective is to minimize the sum of the squared errors

$$\min_{\beta_0, \beta_1} \|\varepsilon\|^2 = \min_{\beta_0, \beta_1} \|Y - X\beta\|^2 = \min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 = \min_{\beta_0, \beta_1} E(\beta_0, \beta_1)$$

We have to find β_0 and β_1 such that:

$$\begin{cases} \frac{\partial E}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0 \\ \frac{\partial E}{\partial \beta_1} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) x_i = 0 \end{cases}$$

$$\begin{cases} \sum_{i=1}^n y_i - n\beta_0 - \beta_1 \sum_{i=1}^n x_i = 0 \\ \sum_{i=1}^n y_i x_i - \beta_0 \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = 0 \end{cases}$$

$$\begin{cases} n\bar{y} - n\beta_0 - n\beta_1 \bar{x} = 0 \\ \sum_{i=1}^n y_i x_i - n\beta_0 \bar{x} - \beta_1 \sum_{i=1}^n x_i^2 = 0 \end{cases}$$

$$\begin{cases} \beta_0 = \bar{y} - \beta_1 \bar{x} \\ \sum_{i=1}^n y_i x_i - n\beta_0 \bar{x} - \beta_1 \sum_{i=1}^n x_i^2 = 0 \end{cases}$$

We plug the first equation in the second one, and we get

$$\sum_{i=1}^n y_i x_i - n\bar{x}(\bar{y} - \beta_1 \bar{x}) - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

$$\sum_{i=1}^n y_i x_i - n\bar{x}\bar{y} + \beta_1 n\bar{x}^2 - \beta_1 \sum_{i=1}^n x_i^2 = 0$$

$$\sum_{i=1}^n y_i x_i - n\bar{x}\bar{y} + \beta_1 (n\bar{x}^2 - \sum_{i=1}^n x_i^2) = 0$$

$$\beta_1 = \frac{\sum_{i=1}^n y_i x_i - n\bar{x}\bar{y}}{\sum_{i=1}^n x_i^2 - n\bar{x}^2}$$

b) Multiple linear regression model

Example: Suppose that Y is the fuel consumption of a particular model of car in m.p.g.

Suppose that the attributes are:

1. X1 — the weight of the car.
2. X2 — the horse power.
3. X3 — the no. of cylinders.

Typically, the data will be available in the form of an array like this:

$$\begin{array}{cccc} y_1 & x_{11} & x_{12} & x_{13} \\ y_2 & x_{21} & x_{22} & x_{23} \\ \dots & & & \\ y_n & x_{n1} & x_{n2} & x_{n3} \end{array}$$

where n is the number of observations or cases in the dataset.

One very general form for the model would be

$$Y = f(X_1, X_2, X_3) + \epsilon$$

ϵ where f is some unknown function and ϵ is the error in this representation which is additive in this instance. Since we usually don't have enough data to try to estimate f directly, we usually have to assume that it has some more restricted form, perhaps linear as

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$$

Matrix Representation

Given the actual data, we may write

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{3i} + \varepsilon_i \quad i = 1, \dots, n$$

But the use of subscripts becomes inconvenient and conceptually obscure. We will find it simpler both notationally and theoretically to use a matrix/vector representation. The regression equation is write

$$y = X\beta + \varepsilon$$

where $y = (y_1, \dots, y_n)^T$, $\beta = (\beta_0, \dots, \beta_3)^T$ and $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^T$

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & x_{13} \\ 1 & x_{21} & x_{22} & x_{23} \\ \dots & & & \\ 1 & x_{n1} & x_{n2} & x_{n3} \end{pmatrix}$$

The column of ones incorporates the intercept term.

Estimating β

The problem is to find β such that $X\beta$ is close to Y . We might define the best estimate of β as that which minimizes the sum of the squared errors, $\varepsilon^T \varepsilon$. That is to say that the least squares estimate of β , called $\hat{\beta}$ minimizes

$$\|\varepsilon\|_2^2 = \varepsilon^T \varepsilon = (y - X\beta)^T (y - X\beta)$$

By applying the transpose operations, we get

$$\begin{aligned} & (y^T - (X\beta)^T)(y - X\beta) \\ & y^T y - y^T X\beta - (X\beta)^T y + (X\beta)^T (X\beta) \\ & y^T y - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta \end{aligned}$$

$y^T X\beta$ is a scalar (a 1x1 matrix), so we apply the symmetry rule: $a^T b = b^T a$

$$\begin{aligned} & y^T y - (X\beta)^T y - \beta^T X^T y + \beta^T X^T X\beta \\ & y^T y - \beta^T X^T y - \beta^T X^T y + \beta^T X^T X\beta \\ & y^T y - 2\beta^T X^T y + \beta^T X^T X\beta \end{aligned}$$

Differentiating with respect to β and setting to zero, we find that $\hat{\beta}_{OLS}$ satisfies

$$\begin{aligned} -2X^T y + 2X^T X\hat{\beta}_{OLS} &= 0 \\ X^T X\hat{\beta}_{OLS} &= X^T y \end{aligned}$$

Provided $X^T X$ is invertible, the OLS solution is

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$$

$X^T X$ is called a Gram matrix, and captures relationships between features.

If collinearity between features is high, then $X^T X$ is almost singular (i.e., its determinant is close to zero). Because the OLS estimates depend upon $(X^T X)^{-1}$, we would have problems in computing $\hat{\beta}_{OLS}$. In such cases, small changes to the elements of X can lead to large variations in $(X^T X)^{-1}$, resulting in inflated coefficients in $\hat{\beta}_{OLS}$. As a result, the OLS estimates become unstable and highly sensitive to small changes in the data.

The variance of the estimated coefficients in Ordinary Least Squares (OLS) is given by:

$$\text{Var}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}$$

- This makes $(X^T X)^{-1}$ very large, which increases the variance of $\hat{\beta}$. Different samples from the data can produce very different estimates of $\hat{\beta}$. The least square estimator $\hat{\beta}$ may provide a good fit to the training data, but it will not fit sufficiently well to the test data.

3.2.2 Regression shrinkage (regularization) methods

3.2.2.1 Ridge Regression

One way out of this situation is to abandon the requirement of an unbiased estimator.

The objective is to minimize the sum of the squared errors

$$\min_{\beta} \|\varepsilon\|^2 = \min_{\beta} \|Y - X\beta\|^2, \text{ s. t. } \|\beta\|^2 \leq c^2$$

Geometric Interpretation of Ridge Regression

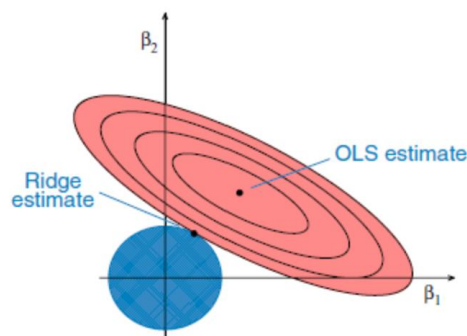


Figure 3.2 Estimation picture of ridge regression. Shown are contours of the error and constraint functions. The solid blue area is the constraint region $\beta_1^2 + \beta_2^2 \leq c^2$, while the red ellipses are the contours of the least squares error function. Ridge shrinks all coefficients but does not eliminate any; coefficients are smaller but nonzero.

The ellipses correspond to the contours of the sum of the squared errors: the inner ellipse has smaller sum of the squared errors, and the sum of the squared errors is minimized at ordinal least square (OLS) estimates.

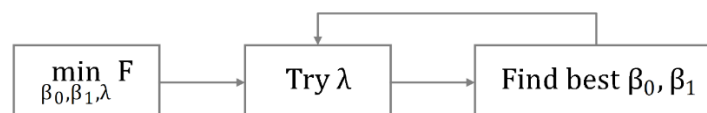
For $p=2$, the constraint in ridge regression corresponds to a circle, $\sum_{j=1}^p \beta_j^2 \leq c^2$

We want to minimize:

$$\min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2, \text{ s. t. } \beta_0^2 + \beta_1^2 \leq c^2$$

Instead of solving this as a constrained optimization problem, it is typically rewritten using a Lagrange multiplier (λ) and solved as an unconstrained optimization problem.

$$\min_{\beta} F(\beta_0, \beta_1, \lambda) = \min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda(\beta_0^2 + \beta_1^2 - c^2)$$



So, we have to minimize

$$\min_{\beta} \|Y - X\beta\|^2 + \lambda \|\beta\|^2$$

The ridge solution

$$\hat{\beta}_{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

3.2.2.2 Lasso Regression

A ridge solution can be hard to interpret because it is not sparse (no β 's are set exactly to 0). What if we constrain the L1 norm instead of the Euclidean (L2) norm?

The objective is to minimize the sum of the squared errors

$$\min_{\beta} \|\varepsilon\|^2 = \min_{\beta} \|Y - X\beta\|^2, s. t. \|\beta\|_1 \leq c$$

For $p=2$, the constraint in ridge regression corresponds to a circle, $\sum_{j=1}^p |\beta_j| \leq c$

We want to minimize:

$$\min_{\beta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2, s. t. |\beta_0| + |\beta_1| \leq c$$

Geometric Interpretation

The lasso performs L1 shrinkage, so that there are "corners" in the constraint, which in two dimensions corresponds to a diamond. If the sum of squares "hits" one of these corners, then the coefficient corresponding to the axis is shrunk to zero.

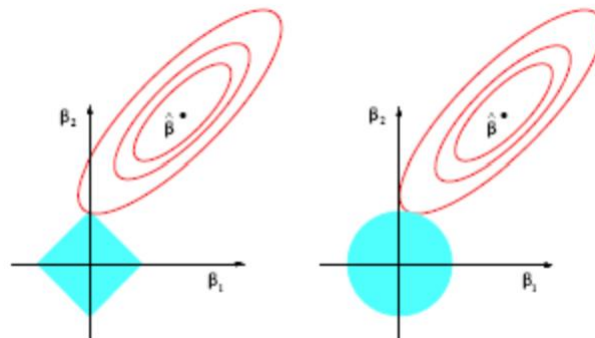


Figure 3.3 Estimation picture of the Lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq c$ and $\beta_1^2 + \beta_2^2 \leq c^2$, respectively, while the red ellipses are the contours of the least squares error function. Lasso shrinks some coefficients to zero, resulting in sparse models.

As p increases, the multidimensional diamond has an increasing number of corners, and so it is highly likely that some coefficients will be set equal to zero. Hence, the lasso performs shrinkage and (effectively) subset selection.

Instead of solving the constrained form, we rewrite it using a Lagrange multiplier (λ):

$$\min_{\beta} \|Y - X\beta\|^2 + \lambda \|\beta\|_1$$

Unlike ridge regression, Lasso does not have a simple closed-form solution because the **L1-norm is not differentiable** at zero. Instead, numerical optimization techniques are used, like gradient descent.

3.2.2.2 Elastic Net Regression

Elastic Net regression combines the strengths of Ridge and Lasso regression, which involves both penalties $L1$ and $L2$. For Elastic Net regression, the objective function is:

$$\min_{\beta} \|Y - X\beta\|^2, s.t. (1 - \rho)\|\beta\|^2 + \rho\|\beta\|_1 \leq c$$

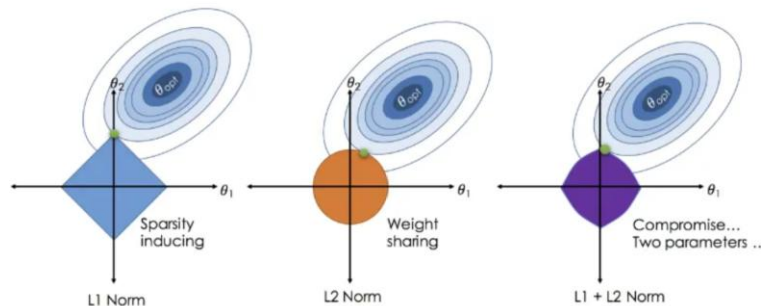


Figure 3.4 Estimation picture of the Lasso (left), ridge regression (middle), and elastic net (right). Shown are contours of the error and constraint functions. In the right, the solid purple area is the constraint region $(1 - \rho)\|\beta\|^2 + \rho\|\beta\|_1 \leq c$. Elastic Net shrinks coefficients like Lasso but distributes the penalty across correlated predictors, often retaining groups of features.

Instead of solving the constrained form, we rewrite it using a Lagrange multiplier (λ):

$$\min_{\beta} \|Y - X\beta\|^2 + \lambda((1 - \rho)\|\beta\|^2 + \rho\|\beta\|_1)$$

Ridge vs. Lasso vs. Elastic Net

Method	Effect on Weak Features	Effect on Collinear Features	Feature Selection?
Lasso (L1)	Eliminates them (sets to zero)	Picks one and removes others	Yes
Ridge (L2)	Shrinks them (but keeps all)	Spreads importance among them	No
Elastic Net	Eliminates weak ones	Keeps multiple collinear ones	Yes (but smoother than Lasso)

Conclusion

Why might we want to use another fitting procedure instead of least squares? As we will see, alternative fitting procedures can yield better prediction accuracy and model interpretability.

- **Prediction Accuracy:** Provided that the true relationship between the response and the predictors is approximately linear, the least squares estimates will have low bias. If $n \gg p$ — that is, if n , the number of observations, is much larger than p , the number of variables — then the least squares estimates tend to also have low variance, and hence will perform well on test observations. However, if n is not much larger than p , then there can be a lot of variability in the least squares fit, resulting in overfitting and consequently poor predictions on future observations not used in model training. And if $p > n$, then there is no longer a unique least squares coefficient estimate: there are infinitely many solutions. Each of these least squares solutions gives zero error on the training data, but typically very poor test set performance due to extremely high variance. By constraining or

shrinking the estimated coefficients, we can often substantially reduce the variance at the cost of a negligible increase in bias. This can lead to substantial improvements in the accuracy with which we can predict the response for observations not used in model training.

- **Model Interpretability:** It is often the case that some or many of the variables used in a multiple regression model are in fact not associated with the response. Including such irrelevant variables leads to unnecessary complexity in the resulting model. By removing these variables — that is, by setting the corresponding coefficient estimates to zero — we can obtain a model that is more easily interpreted. Now least squares is extremely unlikely to yield any coefficient estimates that are exactly zero. Some approaches for automatically performing feature selection or variable selection can be used for excluding irrelevant variables from a multiple regression model.

There are many alternatives, both classical and modern, to using least squares. The three important classes of methods are:

- 1) **Shrinkage:** This approach involves fitting a model involving all p predictors. However, the estimated coefficients are shrunk towards zero relative to the least squares estimates. This shrinkage (also known as regularization) has the effect of reducing variance. Depending on what type of shrinkage is performed, some of the coefficients may be estimated to be exactly zero. Hence, shrinkage methods can also perform variable selection.
- 2) **Subset Selection:** This approach involves identifying a subset of the p predictors that we believe to be related to the response. We then fit a model using least squares on the reduced set of variables.
- 3) **Dimension Reduction:** This approach involves projecting the p predictors into an M -dimensional subspace, where M different linear combinations, or projections, of the variables. Then these M projections are used as predictors to fit a linear regression model by least squares.

3.3 Goodness of Fit

How well does the model fit the data? One measure is R^2 , the so-called coefficient of determination or percentage of variance explained

$$R^2 = \frac{SSE}{SST} = 1 - \frac{SSR}{SST}$$

Total Sum of Squares (SST): Measures the total variation in the dependent variable (y) around its mean ($\bar{y}$)

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

Residual Sum of Squares (SSR): Unexplained variation (error)

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Explained Sum of Squares (SSE): Variation explained by the regression model

$$SSE = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

3.2.2 Iterative or learning-based solution (Gradient Descent)

In supervised learning, we aim to build a model that takes an input x and outputs a prediction y . For simplicity, we assume that both the input x and output y are vectors of a predetermined and fixed size

and that the elements of each vector are always ordered in the same way. This is termed structured or tabular data.

To make the prediction, we need a model $f[\bullet]$ that takes input x and returns y , so:

$$y = f[x].$$

When we compute the prediction y from the input x , we call this inference. The model is just a mathematical equation with a fixed form. It represents a family of different relations between the input and the output. The model also contains parameters ϕ . The choice of parameters determines the particular relation between input and output, so we should really write:

$$y = f[x, \phi].$$

When we talk about learning or training a model, we mean that we attempt to find parameters ϕ that make sensible output predictions from the input. We learn these parameters using a training dataset of I pairs of input and output examples $\{x_i, y_i\}$. We aim to select parameters that map each training input to its associated output as closely as possible. We quantify the degree of mismatch in this mapping with the loss L . This is a scalar value that summarizes how poorly the model predicts the training outputs from their corresponding inputs for parameters ϕ .

We can treat the loss as a function $L[\phi]$ of these parameters. When we train the model, we are seeking parameters $\hat{\phi}$ that minimize this loss function:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L[\phi]].$$

If the loss is small after this minimization, we have found model parameters that accurately predict the training outputs y_i from the training inputs x_i . After training a model, we must now assess its performance; we run the model on separate test data to see how well it generalizes to examples that it didn't observe during training. If the performance is adequate, then we are ready to deploy the model.

3.2 Linear regression example

Let's now make these ideas concrete with a simple example. We consider a model $y = f[x, \phi]$ that predicts a single output y from a single input x . Then we develop a loss function, and finally, we discuss model training.

3.2.1 1D linear regression model

A 1D linear regression model describes the relationship between input x and output y as a straight line:

$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x. \end{aligned}$$

This model has two parameters $\phi = [\phi_0, \phi_1]$, where ϕ_0 is the y-intercept of the line and ϕ_1 is the slope. Different choices for the y-intercept and slope result in different relations between input and output (Figure 3.1). Hence, the above equation defines a family of possible input-output relations (all possible lines), and the choice of parameters determines the member of this family (the particular line).

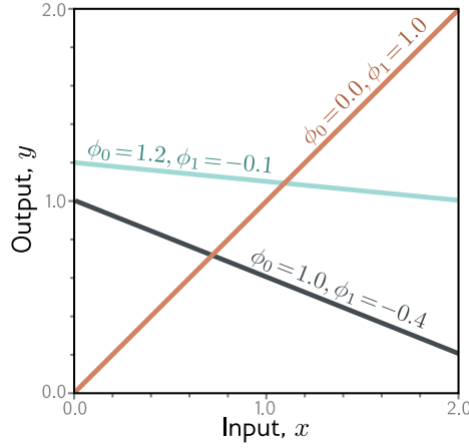


Figure 3.1 Linear regression model.

3.2.2 Loss

For this model, the training dataset (figure 3.2a) consists of I input/output pairs $\{x_i, y_i\}$. Figures 3.2b–d show three lines defined by three sets of parameters. The green line in figure 3.2d describes the data more accurately than the other two since it is much closer to the data points. However, we need a principled approach for deciding which parameters ϕ are better than others. To this end, we assign a numerical value to each choice of parameters that quantifies the degree of mismatch between the model and the data. We term this value the loss; a lower loss means a better fit.

The mismatch is captured by the deviation between the model predictions $f[x_i, \phi]$ (height of the line at x_i) and the ground truth outputs y_i . These deviations are depicted as orange dashed lines in figures 3.2b–d. We quantify the total mismatch, training error, or loss as the sum of the squares of these deviations for all I training pairs:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2. \end{aligned}$$

Since the best parameters minimize this expression, we call this a *least-squares loss*. The squaring operation means that the direction of the deviation (i.e., whether the line is above or below the data) is unimportant.

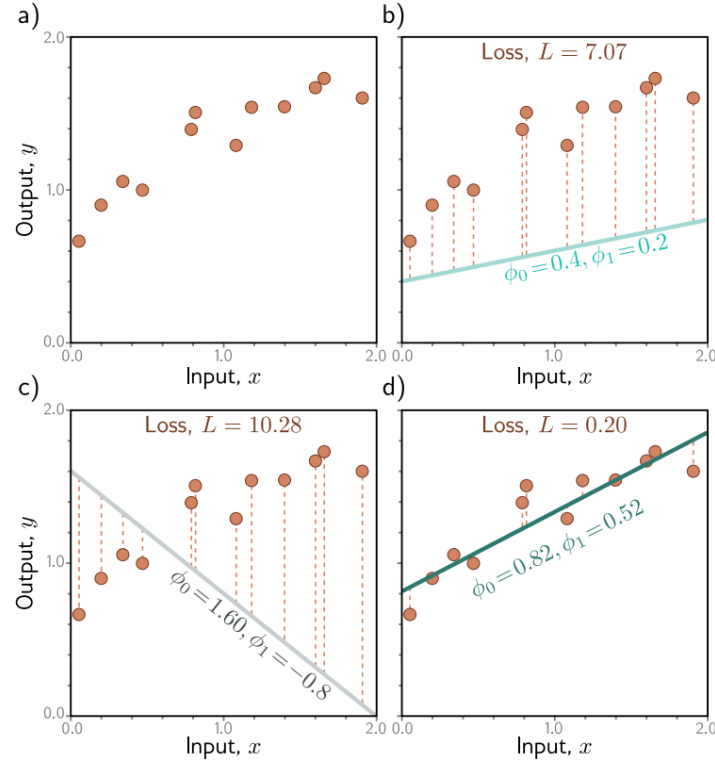


Figure 3.2 Linear regression training data, model, and loss. a) The training data (orange points) consist of $I = 12$ input/output pairs $\{x_i, y_i\}$. b–d) Each panel shows the linear regression model with different parameters. Depending on the choice of y-intercept and slope parameters $\phi = [\phi_0, \phi_1]$, the model errors (orange dashed lines) may be larger or smaller. The loss L is the sum of the squares of these errors. The parameters that define the lines in panels (b) and (c) have large losses $L = 7.07$ and $L = 10.28$, respectively because the models fit badly. The loss $L = 0.20$ in panel (d) is smaller because the model fits well; in fact, this has the smallest loss of all possible lines, so these are the optimal parameters.

The loss L is a function of the parameters ϕ ; it will be larger when the model fit is poor (Figure 3.2b,c) and smaller when it is good (Figure 3.2d). Considered in this light, we term $L[\phi]$ the loss function or cost function. The goal is to find the parameters $\hat{\phi}$ that minimize this quantity:

$$\begin{aligned}\hat{\phi} &= \underset{\phi}{\operatorname{argmin}} [L[\phi]] \\ &= \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \right] \\ &= \underset{\phi}{\operatorname{argmin}} \left[\sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \right].\end{aligned}$$

There are only two parameters (the y-intercept ϕ_0 and slope ϕ_1), so we can calculate the loss for every combination of values and visualize the loss function as a surface (Figure 3.3). The “best” parameters are at the minimum of this surface.

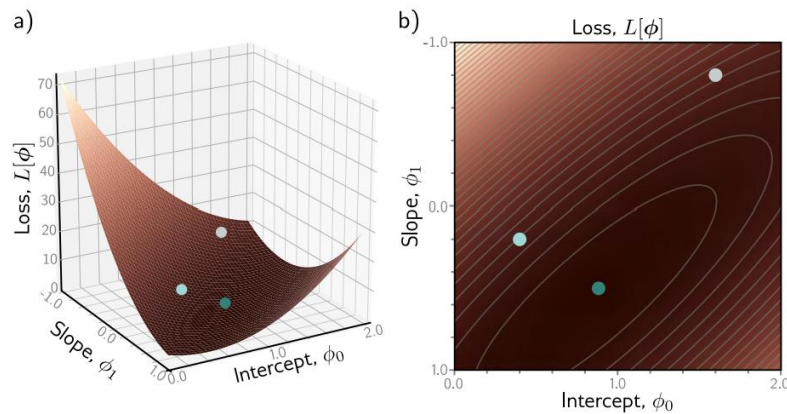


Figure 3.3 Loss function for linear regression model with the dataset in figure 3.2a.

3.2.3 Training

The process of finding parameters that minimize the loss is termed model fitting, training, or learning. The basic method is to choose the initial parameters randomly and then improve them by “walking down” the loss function until we reach the bottom (figure 3.4). One way to do this is to measure the gradient of the surface at the current position and take a step in the direction that is most steeply downhill. Then we repeat this process until the gradient is flat and we can improve no further.

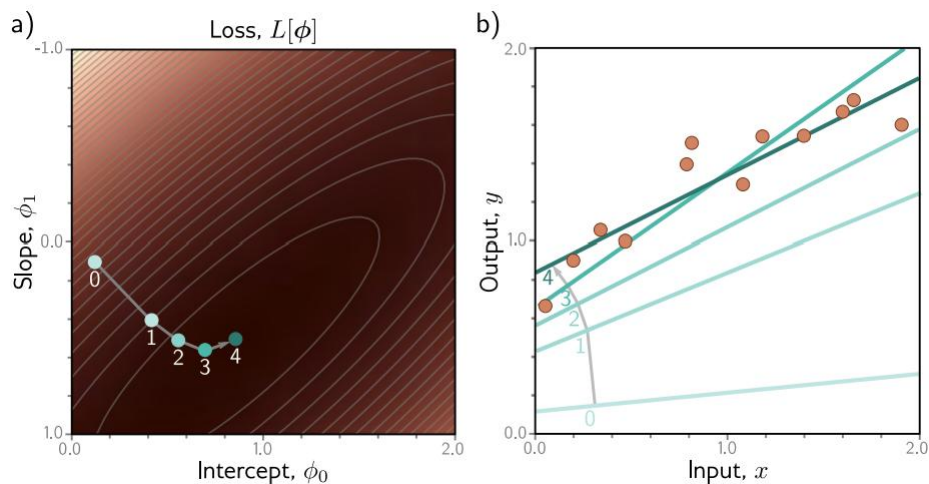


Figure 3.4 Linear regression training.

3.2.4 Testing

Having trained the model, we want to know how it will perform in the real world. We do this by computing the loss on a separate set of test data. The degree to which the prediction accuracy generalizes to the test data depends in part on how representative and complete the training data is. However, it also depends on how expressive the model is. A simple model like a line might not be able to capture the true relationship between input and output. This is known as underfitting. Conversely, a very expressive model may describe statistical peculiarities of the training data that are atypical and lead to unusual predictions. This is known as overfitting.